UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE
Semester Genap 2025/2026

MODUL

Nama Mahasiswa	:	Saprina Melita Sari
NIM	:	23051430012
Kelas	:	A
Tanggal Praktikum	:	3 April 2026
Dosen Pengampu	:	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	MODUL 7 (Pertemuan 7)
Topik Utama	Validation & Local Storage
Sub-Topik	Validasi form tingkat lanjut dan penyimpanan data persisten di browser (Client-side Storage).
Durasi Praktikum	340 Menit
Tanggal Praktikum	3 April 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Menerapkan validasi form sebelum data dikirim untuk mencegah data kotor.
2	Memahami cara kerja Web Storage API (LocalStorage).
3	Mampu menyimpan, mengambil, mengupdate, dan menghapus (CRUD sederhana) data di LocalStorage.
4	Membuat aplikasi "Buku Harian Produksi" sederhana yang datanya tidak hilang saat browser direfresh.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)

5	HTML	-	Struktur
6	CSS		Styling

D. DASAR TEORI

Pengembangan aplikasi web modern mensyaratkan penggunaan teknik validasi form dan penyimpanan data yang efisien di sisi klien agar data yang diolah tetap bersih, akurat, dan persisten. *Validasi form* merupakan proses pemeriksaan input yang dilakukan sebelum data dikirimkan atau diproses lebih lanjut. Tujuan validasi form adalah untuk memastikan bahwa data yang diberikan oleh pengguna telah lengkap, sesuai format, dan bebas dari kesalahan sehingga mencegah terjadinya *data kotor* atau kesalahan dalam pengolahan. Validasi ini dapat dilakukan secara *client-side* menggunakan JavaScript dan HTML5 untuk memberikan respons langsung tanpa harus menunggu proses server, sehingga meningkatkan pengalaman pengguna dan performa aplikasi karena meminimalkan permintaan yang tidak perlu ke server.

Selain validasi form, aplikasi web juga sering membutuhkan mekanisme penyimpanan data agar bisa bertahan antar sesi penggunaan. *Web Storage API,* yang merupakan bagian dari teknologi HTML5, menyediakan dua jenis penyimpanan di sisi klien yaitu *localStorage* dan *sessionStorage*. LocalStorage merupakan fasilitas penyimpanan *key-value* yang memungkinkan data *persisten* tersimpan di browser tanpa batas waktu, bahkan setelah jendela atau tab browser ditutup, sehingga cocok untuk aplikasi yang membutuhkan data tidak hilang saat halaman direfresh atau browser ditutup. Sebaliknya, sessionStorage hanya menyimpan data selama sesi tab aktif berlangsung. LocalStorage memudahkan developer menyimpan, mengambil, memperbarui, dan menghapus data sederhana tanpa perlu backend atau database, sehingga sering digunakan pada aplikasi berskala kecil hingga menengah yang dijalankan sepenuhnya di sisi klien.

Dalam *aplikasi Buku Harian Produksi*, localStorage digunakan untuk menyimpan data produksi harian yang berupa array objek JSON, yang kemudian diolah dan ditampilkan kembali pada antarmuka tabel agar data tidak hilang setelah browser direfresh. Metode ini mengajarkan cara melakukan operasi CRUD sederhana pada data yang tersimpan di localStorage sehingga aplikasi tetap responsif dan data tetap tersedia untuk pengguna *tanpa memerlukan server*. Implementasi ini juga memperkenalkan konsep serialisasi dan deserialisasi objek JavaScript ke format JSON agar data kompleks dapat tersimpan dalam localStorage dengan benar.

D.1 Konsep Utama

Log Produksi Harian mengusung konsep utama pembangunan aplikasi pencatatan produksi yang interaktif, valid, dan persisten di browser**.** Setiap data produksi dimasukkan melalui form HTML yang telah dilengkapi dengan validasi di sisi klien untuk memastikan data lengkap, sesuai format, dan bebas dari kesalahan sebelum disimpan. Data produksi kemudian disimpan secara persisten menggunakan LocalStorage**,** memungkinkan operasi CRUD sederhana sehingga data tetap ada meskipun halaman direfresh. Semua data disimpan sebagai array objek**,** yang dapat dimanipulasi menggunakan fungsi filter untuk menampilkan operator tertentu dan sort untuk mengurutkan jumlah produksi dari terbesar ke terkecil, kemudian ditampilkan secara dinamis di tabel menggunakan manipulasi DOM. Interaktivitas aplikasi juga ditingkatkan melalui fitur reset form setelah submit, tombol hapus data untuk membersihkan LocalStorage dan tabel, serta input pencarian dan tombol sortir yang memudahkan pengguna dalam menganalisis dan meninjau riwayat produksi**.**

D.2 Keterkaitan dengan Teknik Industri

Pencatatan jumlah produksi harian, pemantauan kinerja operator, dan pengelolaan shift kerja merupakan bagian penting dari operasional pabrik untuk meningkatkan efisiensi dan efektivitas produksi. Dengan aplikasi ini, data produksi dapat dianalisis secara cepat, operator yang kurang produktif dapat diidentifikasi, dan keputusan perbaikan proses dapat diambil berdasarkan informasi real-time. Fitur filter dan sortir memudahkan manajer produksi untuk meninjau performa individu atau shift tertentu, sejalan dengan praktik *quality control, performance monitoring, dan production planning* dalam teknik industri.

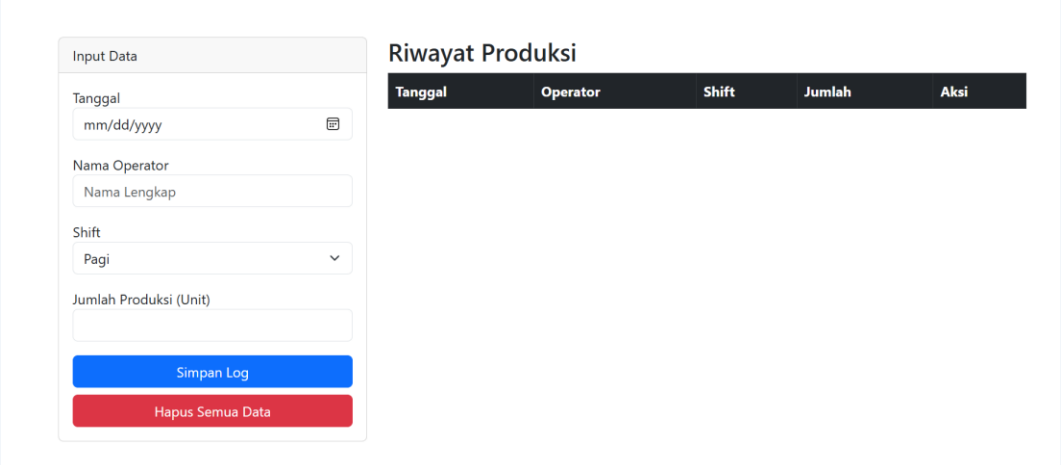
E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Form Input Produksi

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Log Produksi Harian</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"rel="s
</head>
    <body class="container mt-5">

        <div class="row">
            <!-- Kolom Kiri: Form Input -->
             <div class="col-md-4">
                <div class="card">
                    <div class="card-header">Input Data</div>
                    <div class="card-body">
                        <form id="formProduksi">
                            <div class="mb-3">
                                <label>Tanggal</label>
                                <input type="date" id="tanggal" class="form-control"
                            </div>
                            <div class="mb-3">
                                <label>Nama Operator</label>
                                <input type="text" id="operator" class="form-control
placeholder="Nama Lengkap" required>
                            </div>
                            <div class="mb-3">
                                <label>Shift</label>
                                <select id="shift" class="form-select" required>
                                    <option value="Pagi">Pagi</option>
                                    <option value="Siang">Siang</option>
```

```
                                    <option value="Malam">Malam</option>
                                </select>
                            </div>
                            <div class="mb-3">
                                <label>Jumlah Produksi (Unit)</label>
                                <input type="number" id="jumlah" class="form-control
required>
                            </div>
                            <button type="submit" class="btn btn-primary w-100">Simp
Log</button>
                            <button type="button" id="btnHapusSemua" class="btn btn-
100 mt-2">Hapus Semua Data</button>
                        </form>
                    </div>
                </div>
            </div>

            <!-- Kolom Kanan: Tabel Data -->
             <div class="col-md-8">
                <h3>Riwayat Produksi</h3>
                <table class="table table-bordered table-striped">
                    <thead class="table-dark">
                        <tr>
                            <th>Tanggal</th>
                            <th>Operator</th>
                            <th>Shift</th>
                            <th>Jumlah</th>
                            <th>Aksi</th>
                        </tr>
                    </thead>
                    <tbody id="tabelBody">
                        <!-- Data akan masuk di sini lewat JS -->
                        </tbody>
                    </table>
                </div>
            </div>

           <script src="log_produksi.js"></script>
    </body>
</html>
```

Screenshot Hasil:



Gambar E.1 – Hasil Langkah 1

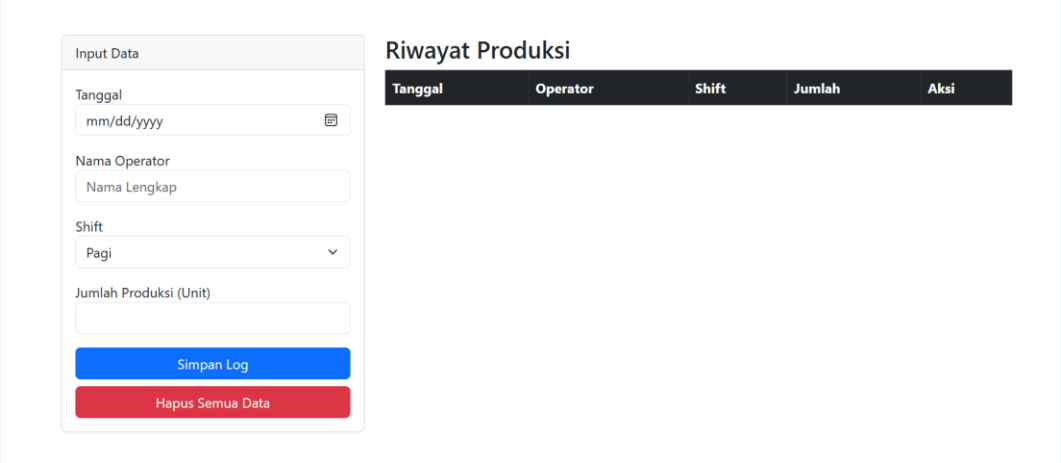
E.2 Langkah 2 – Logika JavaScript (Simpan & Baca)

```
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');

// Kunci untuk LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';

// Fungsi Load Data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function() {
    loadDataFromStorage();
});

// 2. Event Listener: Submit Form
formProduksi.addEventListener('submit', function(event) {
 event.preventDefault(); // Mencegah refresh halaman

 // Ambil Value dari Form
 const tanggal = document.getElementById('tanggal').value;
 const operator = document.getElementById('operator').value;
 const shift = document.getElementById('shift').value;
 const jumlah = document.getElementById('jumlah').value;

 // Validasi Sederhana (JavaScript)
 if (jumlah <= 0) {
 alert("Jumlah produksi harus lebih dari 0!");
 return;
 }
```

```
 // Buat Object Data
 const dataBaru = {
 id: Date.now(), // ID unik berdasarkan waktu
 tanggal: tanggal,
 operator: operator,
 shift: shift,
 jumlah: parseInt(jumlah)
 };

 // Simpan ke LocalStorage
 saveData(dataBaru);

 // Reset Form
 formProduksi.reset();

 // Refresh Tampilan Tabel
 loadDataFromStorage();
});

// 3. Fungsi Simpan ke LocalStorage
function saveData(data) {
 // Ambil data lama (jika ada), jika tidak ada array kosong
 let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

 // Tambah data baru ke array
 dataLama.push(data);

 // Simpan kembali ke LocalStorage (Convert ke JSON String)
 localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}

// 4. Fungsi Baca & Render Tabel
function loadDataFromStorage() {
 // Ambil data
 let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

 // Kosongkan tabel dulu
tabelBody.innerHTML = '';

 // Loop data dan buat baris tabel
 data.forEach(function(item) {
 const row = document.createElement('tr');
 row.innerHTML = `
 <td>${item.tanggal}</td>
 <td>${item.operator}</td>
 <td>${item.shift}</td>
```

```
 <td>${item.jumlah}</td>
 <td>
 <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
 </td>
 `;
 tabelBody.appendChild(row);
 });
}

// 5. Fungsi Hapus Data Spesifik
// Kita pasang di window agar bisa dipanggil dari inline HTML onclick
window.hapusData = function(id) {
 if(confirm('Yakin ingin menghapus log ini?')) {
 let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

 // Filter data: hapus item yang id-nya cocok
 let dataBaru = data.filter(item => item.id !== id);

 // Simpan ulang
 localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

 // Refresh tampilan
 loadDataFromStorage();
 }
}

// 6. Event Hapus Semua
btnHapusSemua.addEventListener('click', function() {
 if(confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
 localStorage.removeItem(STORAGE_KEY);
 loadDataFromStorage();
 }
});
```

Screenshot Hasil:



Gambar E.2 – Hasil Langkah 2

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Filter Data

Tambahkan fitur filter atau pencarian. Buat input "Cari Nama Operator". Saat user mengetik, tabel hanya menampilkan data operator yang namanya cocok (Gunakan `filter` array).

Jawaban / Kode Solusi:

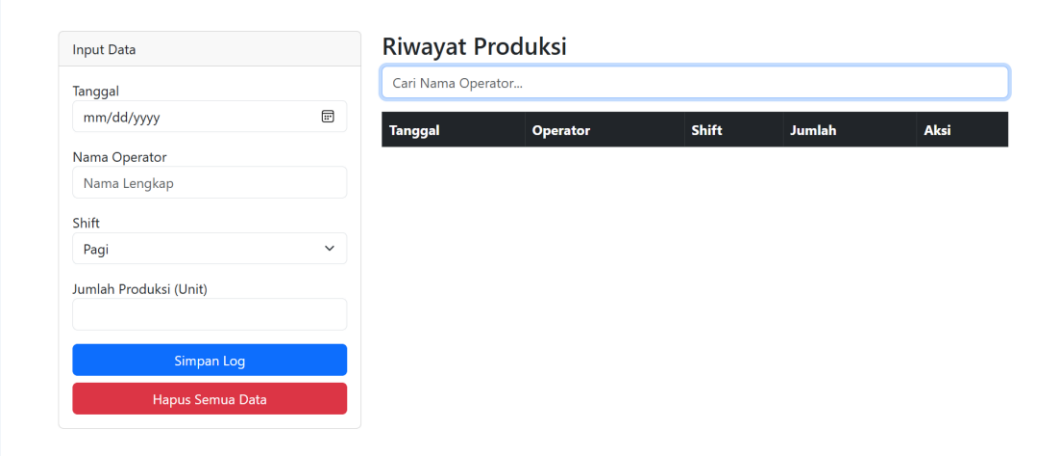
HTML
```
<input type="text" id="inputCari" class="form-control" placeholder="Cari Nama Operator...">
```

JAVASCRIPT
```
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');
// Seleksi elemen inputCari (latihan 1)
const inputCari = document.getElementById('inputCari');
// Seleksi elemen btnSortJumlah (latihan 2)\
const btnSortJumlah = document.getElementById('btnSortJumlah');
// Kunci untuk LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';
// Fungsi Load Data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function () {
    loadDataFromStorage();
});
// 2. Event Listener: Submit Form
```

```javascript
formProduksi.addEventListener('submit', function (event) {
    event.preventDefault(); // Mencegah refresh halaman
    // Ambil Value dari Form
    const tanggal = document.getElementById('tanggal').value;
    const operator = document.getElementById('operator').value;
    const shift = document.getElementById('shift').value;
    const jumlah = document.getElementById('jumlah').value;
    // Validasi Sederhana (JavaScript)
    if (jumlah <= 0) {
        alert("Jumlah produksi harus lebih dari 0!");
        return;
    }
    // Buat Object Data
    const dataBaru = {
        id: Date.now(), // ID unik berdasarkan waktu
        tanggal: tanggal,
        operator: operator,
        shift: shift,
        jumlah: parseInt(jumlah)
    };
    // Simpan ke LocalStorage
    saveData(dataBaru);
    // Reset Form
    formProduksi.reset();

    // Refresh Tampilan Tabel
    loadDataFromStorage();
});
// 3. Fungsi Simpan ke LocalStorage
function saveData(data) {
    // Ambil data lama (jika ada), jika tidak ada array kosong
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Tambah data baru ke array
    dataLama.push(data);

    // Simpan kembali ke LocalStorage (Convert ke JSON String)
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}
// 4. Fungsi Baca & Render Tabel
function loadDataFromStorage() {
    // Ambil data
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Kosongkan tabel dulu
    tabelBody.innerHTML = '';
```

```
    // Loop data dan buat baris tabel
    data.forEach(function (item) {
        const row = document.createElement('tr');
        row.innerHTML = `
 <td>${item.tanggal}</td>
 <td>${item.operator}</td>
 <td>${item.shift}</td>
 <td>${item.jumlah}</td>
 <td>
 <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
 </td>
 `;
        tabelBody.appendChild(row);
    });
}
// 5. Fungsi Hapus Data Spesifik
// Kita pasang di window agar bisa dipanggil dari inline HTML onclick
window.hapusData = function (id) {
    if (confirm('Yakin ingin menghapus log ini?')) {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

        // Filter data: hapus item yang id-nya cocok
        let dataBaru = data.filter(item => item.id !== id);

        // Simpan ulang
        localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

        // Refresh tampilan
        loadDataFromStorage();
    }
}
// 6. Event Hapus Semua
btnHapusSemua.addEventListener('click', function () {
    if (confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
        localStorage.removeItem(STORAGE_KEY);
        loadDataFromStorage();
    }
});
// membuat function renderTable untuk menampilkan hasil filter
inputCari(latihan 1)
function renderTable(data) {
    tabelBody.innerHTML = '';

    data.forEach(function (item) {
        const row = document.createElement('tr');
```

```
        row.innerHTML = `
            <td>${item.tanggal}</td>
            <td>${item.operator}</td>
            <td>${item.shift}</td>
            <td>${item.jumlah}</td>
            <td>
                <button class="btn btn-sm btn-danger"
                onclick="hapusData(${item.id})">Hapus</button>
            </td>
        `;
        tabelBody.appendChild(row);
    });
}

// Membuat evenListener inputCari untuk filter nama operator (latihan 1)
inputCari.addEventListener('input', function () {
    const keyword = this.value.toLowerCase();
    const data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    const hasilFilter = data.filter(item =>
        item.operator.toLowerCase().includes(keyword)
    );

    renderTable(hasilFilter);
});

// Membuat evenListener btnSortJumlah untuk mengurutkan data (latihan 2)
btnSortJumlah.addEventListener('click', function () {
    const keyword = inputCari.value.toLowerCase();
    const data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
    // Filter dulu sesuai kondisi input saat ini
    const hasilFilter = data.filter(item =>
        item.operator.toLowerCase().includes(keyword)
    );
    // Urutkan hasil filter
    const dataUrut = hasilFilter.sort((a, b) => b.jumlah - a.jumlah);

    renderTable(dataUrut);
});
```



Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 –

Tambahkan tombol "Sortir berdasarkan Jumlah (Terbesar)" yang mengurutkan tabel dari jumlah produksi terbanyak ke terkecil.

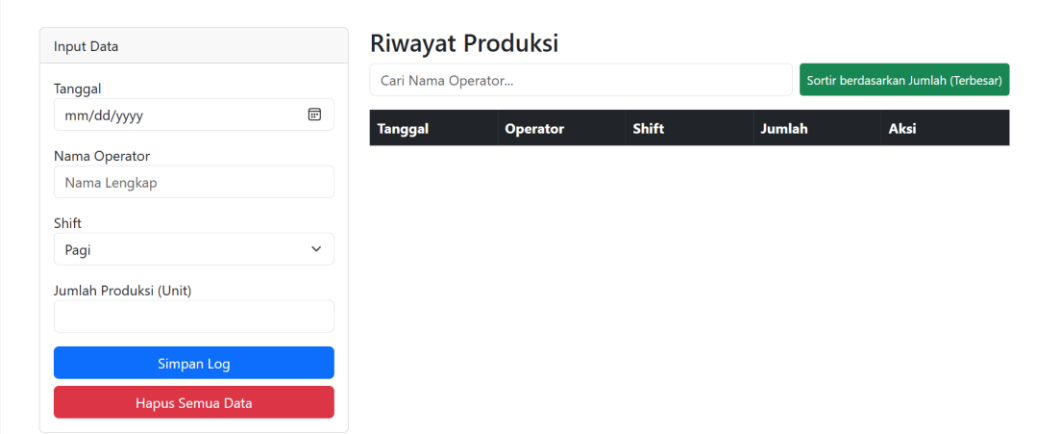
Jawaban / Kode Solusi:

```
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');
// Seleksi elemen inputCari (latihan 1)
const inputCari = document.getElementById('inputCari');
// Seleksi elemen btnSortJumlah (latihan 2)\
const btnSortJumlah = document.getElementById('btnSortJumlah');
// Kunci untuk LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';
// Fungsi Load Data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function () {
    loadDataFromStorage();
});
// 2. Event Listener: Submit Form
```

```javascript
formProduksi.addEventListener('submit', function (event) {
    event.preventDefault(); // Mencegah refresh halaman
    // Ambil Value dari Form
    const tanggal = document.getElementById('tanggal').value;
    const operator = document.getElementById('operator').value;
    const shift = document.getElementById('shift').value;
    const jumlah = document.getElementById('jumlah').value;
    // Validasi Sederhana (JavaScript)
    if (jumlah <= 0) {
        alert("Jumlah produksi harus lebih dari 0!");
        return;
    }
    // Buat Object Data
    const dataBaru = {
        id: Date.now(), // ID unik berdasarkan waktu
        tanggal: tanggal,
        operator: operator,
        shift: shift,
        jumlah: parseInt(jumlah)
    };
    // Simpan ke LocalStorage
    saveData(dataBaru);
    // Reset Form
    formProduksi.reset();

    // Refresh Tampilan Tabel
    loadDataFromStorage();
});
// 3. Fungsi Simpan ke LocalStorage
function saveData(data) {
    // Ambil data lama (jika ada), jika tidak ada array kosong
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Tambah data baru ke array
    dataLama.push(data);

    // Simpan kembali ke LocalStorage (Convert ke JSON String)
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}
// 4. Fungsi Baca & Render Tabel
function loadDataFromStorage() {
    // Ambil data
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Kosongkan tabel dulu
    tabelBody.innerHTML = '';
```

```
    // Loop data dan buat baris tabel
    data.forEach(function (item) {
        const row = document.createElement('tr');
        row.innerHTML = `
 <td>${item.tanggal}</td>
 <td>${item.operator}</td>
 <td>${item.shift}</td>
 <td>${item.jumlah}</td>
 <td>
 <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
 </td>
 `;
        tabelBody.appendChild(row);
    });
}
// 5. Fungsi Hapus Data Spesifik
// Kita pasang di window agar bisa dipanggil dari inline HTML onclick
window.hapusData = function (id) {
    if (confirm('Yakin ingin menghapus log ini?')) {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

        // Filter data: hapus item yang id-nya cocok
        let dataBaru = data.filter(item => item.id !== id);

        // Simpan ulang
        localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

        // Refresh tampilan
        loadDataFromStorage();
    }
}
// 6. Event Hapus Semua
btnHapusSemua.addEventListener('click', function () {
    if (confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
        localStorage.removeItem(STORAGE_KEY);
        loadDataFromStorage();
    }
});
// membuat function renderTable untuk menampilkan hasil filter
inputCari(latihan 1)
function renderTable(data) {
    tabelBody.innerHTML = '';

    data.forEach(function (item) {
        const row = document.createElement('tr');
```

```
        row.innerHTML = `
            <td>${item.tanggal}</td>
            <td>${item.operator}</td>
            <td>${item.shift}</td>
            <td>${item.jumlah}</td>
            <td>
                <button class="btn btn-sm btn-danger"
                onclick="hapusData(${item.id})">Hapus</button>
            </td>
        `;
        tabelBody.appendChild(row);
    });
}

// Membuat evenListener inputCari untuk filter nama operator (latihan 1)
inputCari.addEventListener('input', function () {
    const keyword = this.value.toLowerCase();
    const data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    const hasilFilter = data.filter(item =>
        item.operator.toLowerCase().includes(keyword)
    );

    renderTable(hasilFilter);
});

// Membuat evenListener btnSortJumlah untuk mengurutkan data (latihan 2)
btnSortJumlah.addEventListener('click', function () {
    const keyword = inputCari.value.toLowerCase();
    const data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
    // Filter dulu sesuai kondisi input saat ini
    const hasilFilter = data.filter(item =>
        item.operator.toLowerCase().includes(keyword)
    );
    // Urutkan hasil filter
    const dataUrut = hasilFilter.sort((a, b) => b.jumlah - a.jumlah);

    renderTable(dataUrut);
});
```



Gambar F.2 – Hasil Latihan 2

F.3 Tugas Proyek Mini – Aplikasi Ckecklist 5S

Deskripsi proyek mini:

Buat aplikasi checklist sederhana audit 5S (Seiri, Seiton, Seiso, Seiketsu, Shitsuke).
a. Form berisi 5 checkbox (boolean) dan input nama auditor.
b. Saat disimpan, hitung skor (jumlah checklist dicentang / 5 * 100).
c. Simpan riwayat audit ke LocalStorage dan tampilkan di tabel (Tanggal, Auditor, Skor).

Penjelasan pendekatan/solusi yang digunakan:

1. Penyimpanan Data
   Solusi: Riwayat audit disimpan di browser agar tetap ada meskipun halaman direfresh.
   Kode/Teknis: Menggunakan LocalStorage (localStorage.setItem / localStorage.getItem) untuk menyimpan array objek JSON.
2. Form Input Dinamis
   Solusi: Form digunakan untuk memasukkan nama auditor dan menandai checklist 5S.
   Kode/Teknis: Menggunakan HTML form dan input type="text" untuk nama auditor, input type="checkbox" untuk checklist.
3. Pengambilan Status Checklist
   Solusi: Status checkbox dibaca sebagai benar/salah untuk menghitung skor.
   Kode/Teknis: Menggunakan properti .checked di checkbox, menghasilkan tipe Boolean (true/false).
4. Perhitungan Skor Otomatis
   Solusi: Skor dihitung dari jumlah checklist yang dicentang dibagi 5, dikonversi ke persen.

Kode/Teknis: Array checklist di-filter untuk menghitung yang true, hasilnya dikalikan 100 → tipe data Number.

5. Penambahan Data Baru
   Solusi: Data audit baru ditambahkan ke riwayat dan disimpan kembali di LocalStorage.
   Kode/Teknis: Menggunakan array push() untuk menambah objek {tanggal, auditor, skor} ke array dataAudit, lalu disimpan dengan LocalStorage.
6. Tabel Riwayat Dinamis
   Solusi: Tabel audit diperbarui otomatis saat ada data baru atau dihapus.
   Kode/Teknis: Membuat fungsi renderTabel(), menggunakan DOM manipulation (document.createElement, .innerHTML) untuk menampilkan array data.
7. Interaktivitas Form
   Solusi: Form otomatis direset setelah submit agar siap input baru.
   Kode/Teknis: Memanggil form.reset() setelah menambah data → membersihkan input teks dan checkbox.
8. Penghapusan Data
   Solusi: Ada tombol untuk menghapus semua data audit sekaligus, membersihkan LocalStorage dan tabel.
   Kode/Teknis: Mengosongkan array dataAudit = [], localStorage.removeItem(), lalu memanggil renderTabel() untuk update tampilan.

Kode Utama:

HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Audit 5S</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">

    <h2>Checklist Audit 5S</h2>
    <div class="row">
        <div class="col-md-4">
            <div class="card">
                <div class="card-header">Form Audit</div>
                <div class="card-body">
                    <form id="formAudit">
                        <div class="mb-3">
                            <label>Nama Auditor</label>
                            <input type="text" id="auditor" class="form-
control" placeholder="Masukkan Nama" required>
                        </div>
                        <div class="mb-3">
                            <div class="form-check">
```

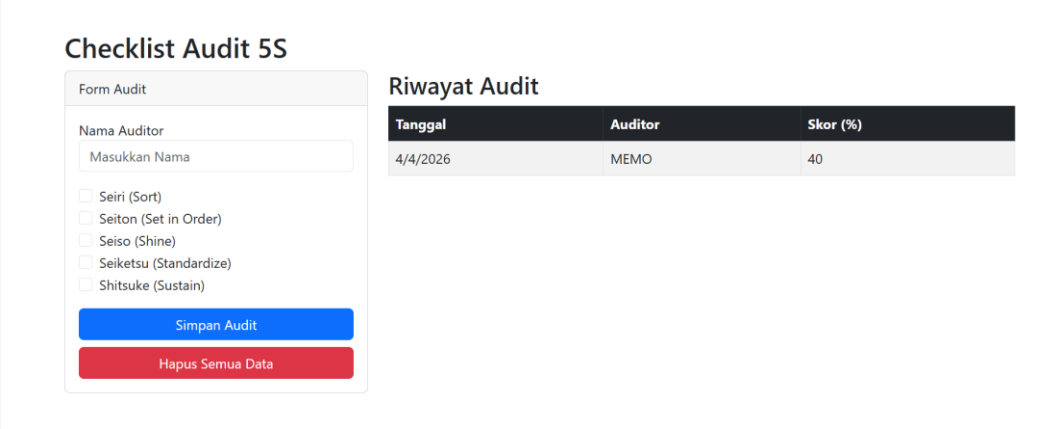
```
                                <input type="checkbox" class="form-check-
input" id="seiri">
                                <label class="form-check-label"
for="seiri">Seiri (Sort)</label>
                            </div>
                            <div class="form-check">
                                <input type="checkbox" class="form-check-
input" id="seiton">
                                <label class="form-check-label"
for="seiton">Seiton (Set in Order)</label>
                            </div>
                            <div class="form-check">
                                <input type="checkbox" class="form-check-
input" id="seiso">
                                <label class="form-check-label"
for="seiso">Seiso (Shine)</label>
                            </div>
                            <div class="form-check">
                                <input type="checkbox" class="form-check-
input" id="seiketsu">
                                <label class="form-check-label"
for="seiketsu">Seiketsu (Standardize)</label>
                            </div>
                            <div class="form-check">
                                <input type="checkbox" class="form-check-
input" id="shitsuke">
                                <label class="form-check-label"
for="shitsuke">Shitsuke (Sustain)</label>
                            </div>
                        </div>
                        <button type="submit" class="btn btn-primary w-
100">Simpan Audit</button>
                        <button type="button" id="btnHapusSemua" class="btn
btn-danger w-100 mt-2">Hapus Semua Data</button>
                    </form>
                </div>
            </div>
        </div>

        <div class="col-md-8">
            <h3>Riwayat Audit</h3>
            <table class="table table-bordered table-striped">
                <thead class="table-dark">
                    <tr>
                        <th>Tanggal</th>
                        <th>Auditor</th>
```

```
                        <th>Skor (%)</th>
                    </tr>
                </thead>
                <tbody id="tabelAudit">
                    <!-- Data akan muncul di sini -->
                </tbody>
            </table>
        </div>
    </div>

    <script src="aplikasi checklist.js"></script>
</body>
</html>
```

JAVASCRIPT

```
// ======================
// AUDIT 5S
// ======================

// 1. Ambil elemen
const formAudit = document.getElementById('formAudit');
const tabelAudit = document.getElementById('tabelAudit');
const btnHapusSemua = document.getElementById('btnHapusSemua');

// 2. Ambil data dari LocalStorage atau inisialisasi kosong
let dataAudit = JSON.parse(localStorage.getItem('dataAudit')) || [];

// 3. Fungsi render tabel
function renderTabel() {
    tabelAudit.innerHTML = '';
    dataAudit.forEach((item, index) => {
        const row = document.createElement('tr');
        row.innerHTML = `
            <td>${item.tanggal}</td>
            <td>${item.auditor}</td>
            <td>${item.skor}</td>
        `;
        tabelAudit.appendChild(row);
    });
}

// 4. Render tabel pertama kali
renderTabel();

// 5. Event submit form
formAudit.addEventListener('submit', function(e) {
```

```
    e.preventDefault();

    const auditor = document.getElementById('auditor').value;
    const seiri = document.getElementById('seiri').checked;
    const seiton = document.getElementById('seiton').checked;
    const seiso = document.getElementById('seiso').checked;
    const seiketsu = document.getElementById('seiketsu').checked;
    const shitsuke = document.getElementById('shitsuke').checked;

    // Hitung skor
    const total = [seiri, seiton, seiso, seiketsu, shitsuke].filter(v =>
v).length;
    const skor = Math.round((total / 5) * 100);

    // Tambahkan ke array
    const tanggal = new Date().toLocaleDateString('id-ID');
    dataAudit.push({ tanggal, auditor, skor });

    // Simpan ke LocalStorage
    localStorage.setItem('dataAudit', JSON.stringify(dataAudit));

    // Render tabel
    renderTabel();

    // Reset form
    formAudit.reset();
});

// 6. Hapus semua data
btnHapusSemua.addEventListener('click', () => {
    if (confirm('Apakah yakin ingin menghapus semua data audit?')) {
        dataAudit = [];
        localStorage.removeItem('dataAudit');
        renderTabel();
    }
});
```



Gambar F.3 – Hasil Proyek Mini

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Form input dibuat menggunakan HTML dengan tipe data yang sesuai, yaitu tanggal, teks, select, dan number, sehingga validasi input bisa dilakukan dengan benar. Data produksi disimpan dalam bentuk array objek, yang memungkinkan manipulasi lebih lanjut seperti menambah data baru, menghapus, maupun menyimpan riwayat secara persisten menggunakan LocalStorage. Untuk latihan 1, fitur filter data menggunakan Array.filter(), sehingga ketika pengguna mengetik nama operator, tabel hanya menampilkan data yang sesuai, sesuai teori filter array. Sedangkan pada latihan 2, tombol sortir mengurutkan data dari jumlah produksi terbesar ke terkecil menggunakan Array.sort(), yang juga sesuai teori pengurutan data numerik. Tabel riwayat produksi diperbarui secara dinamis menggunakan manipulasi DOM, sehingga setiap perubahan data—baik penambahan, filter, maupun sortir—langsung tercermin di tampilan tabel.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Data tidak tersimpan setelah halaman direfresh.	Menggunakan LocalStorage untuk menyimpan array data audit sehingga tetap ada meskipun halaman dimuat ulang.
Filter data operator tidak berjalan saat mengetik karena tabel tidak update.	Menggunakan Array.filter() untuk memilih data yang sesuai input dan render ulang tabel setiap kali input berubah.
Data tidak bisa diurutkan dari jumlah terbesar ke terkecil.	Menggunakan Array.sort() pada properti jumlah produksi dan render ulang tabel setelah sortir.

G.3 Kaitan dengan Penerapan di Industri

Log Produksi Harian memiliki kaitan yang erat dengan industri manufaktur dan produksi**,** di mana pencatatan jumlah produksi harian, pemantauan kinerja operator, dan pengelolaan shift kerja merupakan bagian penting dari operasional pabrik. Dengan aplikasi ini, manajemen dapat memantau produktivitas secara real-time, mengidentifikasi operator atau shift dengan performa tinggi atau rendah, serta memudahkan pengambilan keputusan berbasis data. Selain itu, fitur filter dan sortir memudahkan analisis data produksi, misalnya menemukan operator tertentu atau melihat unit produksi terbesar, yang sangat relevan dengan praktik industrial engineering, quality control, dan performance monitoring**.**

H. KESIMPULAN

1	Form input divalidasi di sisi klien untuk memastikan data lengkap dan bersih, sementara LocalStorage digunakan agar data produksi tetap tersimpan secara persisten.
2	Data disimpan sebagai array objek yang dapat difilter dan diurutkan; tabel riwayat produksi diperbarui secara dinamis, memudahkan analisis operator dan jumlah produksi.
3	Aplikasi meniru praktik pencatatan produksi di dunia nyata, mendukung pemantauan kinerja, quality control, dan pengambilan keputusan berbasis data di lingkungan manufaktur dan teknik industri.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
1.	Al-Rousan, Thamer, et al. *"A New Security Model for Web Browser Local Storage." International Journal of Advanced Computer Science and Applications (IJACSA)*, Vol. 10, No. 8, 2019.
2.	Sinha, Akash Rakesh. *"Navigating Client-Side Storage in Modern Web Applications: Mechanisms, Best Practices and Future Directions." International Journal for Multidisciplinary Research (IJFMR)*, Vol. 2, Issue 5, September–October 2020.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
	TOTAL NILAI AKHIR	100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
Saprina Melita Sari NIM: 23051430012	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____